COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/16/2026
NAME:	Juan Paulo C. Lara Mencidor, Renverlanz D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

Strategy	Algorithm	Analysis
Recursion	Recursive Power Function (from HOA 11)	(LARA) This recursive function rpower() performs its recursion through rpower(x, n - 1) to reach the conditional case n == 0, and therefore return the value 1. For each time this function calls itself, it performs a reduction on the exponent until this function satisfies the condition and returns the value. (MENCIDOR) The recursive power function exemplifies the Recursion strategy by computing x^n through repeated self referential calls with decremented or halved exponents until hitting the base case (x^0 = 1). By delegating work to smaller sub instances of the same function and resolving them as the call stack unwinds, it eliminates the need for explicit iterative looping structures.
Brute Force	Bubble Sort Algorithm	(LARA) This algorithm contains two nested loops where each loop compares one element with other unsorted elements in the array. It typically has a default condition where each of the next elements must be greater than the previous ones. As it repeatedly compares and swaps the elements in the list in order, it

		pushes the larger values to the right. (MENCIDOR) Bubble Sort relies on the Brute Force strategy by systematically checking and swapping adjacent elements across *n* complete passes over an array. It avoids complex optimization logic or advanced spatial partitioning, instead relying on raw, exhaustive comparison iterations to guarantee that unsorted elements gradually drift into their correct position.
Backtracking	Depth-First Search (DFS)	(LARA) This searching algorithm traverses and visits the vertices in the graph downwards until it reaches the end point, which then performs its backtracking by heading back to the previous vertices before the dead end. It uses stacks as its memory of the vertices visited and pops the current node and returns to the previous node to enter the unvisited branches. (MENCIDOR) DFS represents the Backtracking paradigm by exploring a graph or tree state space as deeply as possible along a single path before hitting a dead end. Once an unviable node or fully explored branch is reached, the algorithm unwinds back to the previous decision junction to evaluate unexplored alternative paths.
Greedy	Selection Sort	(LARA) This sorting algorithm reads the unsorted sub-arrays to make an optimal choice of finding the minimum element and immediately swapping it into its correct position. It does not perform backtracking or read any other position other than the one it is currently sorting. (MENCIDOR)

Divide-and-Conquer	Merge Sort	(LARA) This algorithm performs the Divide-and-Conquer strategy by obtaining the midpoint of the array and splitting the main array into two sub-arrays (Divide). Then, it recursively sorts the left and right halves of the array, which divides the sub-arrays into smaller ones (Divide). After this, it invokes a helper function merge() to compare and combine the divided and sorted sub-arrays into one fully sorted array (Conquer). (MENCIDOR) It embodies the Divide-and-Conquer strat by splitting an unsorted list into two equal sub arrays (*Divide),* recursively sorting each half (*Conquer),* and merging the two sorted sub lists back into a single ordered array(*Combine*). This approach guarantees a stable and consistent O(n log n) time complexity across all input configs.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	Code

```cpp
#include <iostream>
#include <algorithm>

int memo[100];

int getMinStepsMemo ( int n )
{
    if ( n == 1 ) return 0; // base case
    if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinStepsMemo( n - 1 ); // '-1' step . 'r' will contain the optimal answer
    finally
    if( n%2 == 0 ) r = std::min( r , 1 + getMinStepsMemo( n / 2 ) ) ; // '/2' step
    if( n%3 == 0 ) r = std::min( r , 1 + getMinStepsMemo( n / 3 ) ) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}

int main() {
    int testCases[] = {1, 4, 7, 10};

    std::cout << "=== Table 12-2: Memoization ===" << std::endl;
    for (int n : testCases) {
        // Initialize memo array with -1 (-1 means not solved yet)
        for (int i = 0; i <= n; i++) {
            memo[i] = -1;
        }

        std::cout << "n = " << n << " -> Minimum Steps: " << getMinStepsMemo(n) << std::endl;
    }

    return 0;
}
```

Output

```
=== Table 12-2: Memoization ===
n = 1 -> Minimum Steps: 0
n = 4 -> Minimum Steps: 2
n = 7 -> Minimum Steps: 3
n = 10 -> Minimum Steps: 3
```

Analysis

(LARA)
This code implementation uses a Top-down dynamic programming method called memoization to solve the minimum steps to reduce N to 1 through computing the least amount of operations to reduce the given integer to 1 by subtracting, dividing by 2, and dividing by 3. Its base case is 1, which returns zero since 1 is already reached. It contains a memo array where the uncomputed stages are set to -1 so the function can return a cached result to reduce the function calls. This uses std::min to find the minimum steps for the three methods of reducing the numbers. With memoization, the exponential $O(3^n)$ time complexity for a brute force method is reduced to an O(n) linear process as each integer up to n is calculated at most once. The space complexity is O(n) due to an array storing a cache of operations and a recursive call stack.

(MENCIDOR)
This applies a Top-down Dynamic Programming approach using memoization to compute the

	minimum steps needed to reduce N down to 1 via subtracting 1, dividing by 2, or dividing by 3. The base case is set at 1, which immediately returns 0 operations. To avoid repeating expensive function calls, the memo array initializes uncalculated states to -1 so the program can quickly return cached results. The code uses std::min to evaluate and pick the shortest path among the three available choices. By storing prior results, memoization dramatically cuts the exponential O(3^n) brute-force runtime down to a linear O(n) process, processing each integer up to N at most once. Its space complexity is O(n), driven by the memory needed for the lookup cache and the recursive call stack.

Table 12-2. Memoization Implementation

Screenshot	Code
	`#include <iostream>`
	`#include <algorithm>`
	
	`int getMinStepsBU ( int n )`
	`{`
	`  int dp[100], i;`
	`  dp[1] = 0; // trivial case`
	
	`  for( i = 2 ; i <= n ; i ++ )`
	`  {`
	`  dp[i] = 1 + dp[i-1];`
	`  if(i%2==0) dp[i] = std::min( dp[i] , 1+ dp[i/2] );`
	`  if(i%3==0) dp[i] = std::min( dp[i] , 1+ dp[i/3] );`
	`  }`
	`  return dp[n];`
	`}`
	
	`int main() {`
	`  int testCases[] = {1, 4, 7, 10};`
	
	`  std::cout << "=== Table 12-3: Bottom-Up ===" << std::endl;`
	`  for (int n : testCases) {`
	`  // Initialize memo array with -1 (-1 means not solved yet)`
	`  std::cout << "n = " << n << " -> Minimum Steps: " << getMinStepsBU(n) << std::endl;`
	`  }`
	
	`  return 0;`
	`}`
	
	Output
	`=== Table 12-3: Bottom-Up ===`
	`n = 1 -> Minimum Steps: 0`
	`n = 4 -> Minimum Steps: 2`
	`n = 7 -> Minimum Steps: 3`
	`n = 10 -> Minimum Steps: 3`

Analysis	(LARA) In contrast to memoization, this tabulation approach to dynamic programming uses iteration to solve the minimum steps of reducing n to 1. This also computes the least amount of operations to reduce the given integers to 1 by using the same three methods used in the previous

	implementation. In this code, it initializes the starting case as dp[1] = 0 for setting the operations required for n = 1 as zero. It then uses loops starting from i = 2 to n to calculate and store the optimal solutions in the array. For the methods, it sets dp[i] = 1 + dp[i-1] as the baseline and uses std::min to update the array. i/2 and i/3 are set to lower the count of steps for reduction. Compared to the previous method, it has less overhead in recursion and less chances of a stack overflow as it performs the process using loops. It has a time complexity of O(n) since the loops execute n -1 times to form the data table. Similarly, it also has a space complexity of O(n) for the values stored in the array. (MENCIDOR) Unlike top-down memoization, this tabulation method takes an iterative, bottom-up approach to compute the fewest operations needed to reduce N to 1 using the same three rules. The program sets dp[1] = 0 as its starting baseline since N = 1 requires zero steps, then runs a loop from i = 2 up to n to build out optimal answers in the array. It defaults to dp[i] = 1 + dp[i-1] and updates the entry using std::min whenever dividing by 2 or 3 yields a shorter path. Because it relies entirely on loops rather than recursive calls, this iterative dynamic programming strategy eliminates function call overhead and prevents stack overflow issues. The time complexity is O(n) since the loop executes n - 1 times to populate the table, and the space complexity stays at O(n) to store the DP array.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B

Pseudocode

```
FUNCTION countPaths(mat, M, N, i, j, cost)
   IF i >= M OR j >= N THEN
      RETURN 0
   ENDIF

   IF i == M - 1 AND j == N - 1 THEN
      IF cost == mat[i][j] THEN
         RETURN 1
      ELSE
         RETURN 0
      ENDIF
   ENDIF

   INITIALIZE remainingCost = cost - mat[i][j]
```

```
    INITIALIZE rightPaths = countPaths(mat, M, N, i, j + 1, remainingCost)
    INITIALIZE downPaths = countPaths(mat, M, N, i + 1, j, remainingCost)

    RETURN rightPaths + downPaths
ENDFUNCTION

FUNCTION MAIN()
    INITIALIZE mat = [
        [4, 7, 1, 6],
        [6, 7, 3, 9],
        [3, 8, 1, 2],
        [7, 1, 7, 3]
    ]

    INITIALIZE M = ROW_COUNT(mat)
    INITIALIZE N = COLUMN_COUNT(mat)

    INITIALIZE cost1 = 25
    INITIALIZE cost2 = 26
    INITIALIZE cost3 = 31

    INITIALIZE result1 = countPaths(mat, M, N, 0, 0, cost1)
    INITIALIZE result2 = countPaths(mat, M, N, 0, 0, cost2)
    INITIALIZE result3 = countPaths(mat, M, N, 0, 0, cost3)

    PRINT "Cost: " + cost1
    PRINT "Paths: " + result1

    PRINT "Cost: " + cost2
    PRINT "Paths: " + result2

    PRINT "Cost: " + cost3
    PRINT "Paths: " + result3

    RETURN 0
ENDFUNCTION
```

Working Demonstration

Code

```cpp
#include <iostream>
#include <vector>

int countPaths(const std::vector<std::vector<int>>& mat, int M, int N, int i, int j, int cost) {
    if (i >= M || j >= N) {
        return 0;
    }

    if (i == M - 1 && j == N - 1) {
        if (cost == mat[i][j]) {
            return 1;
        }
        return 0;
    }

    int remainingCost = cost - mat[i][j];

    int rightPaths = countPaths(mat, M, N, i, j + 1, remainingCost);
    int downPaths = countPaths(mat, M, N, i + 1, j, remainingCost);

    return rightPaths + downPaths;
}
```

```cpp
int main() {
    std::vector<std::vector<int>> mat = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int M = mat.size();
    int N = mat[0].size();

    int cost1 = 25;
    int cost2 = 26;
    int cost3 = 31;

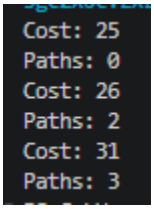
    int result1 = countPaths(mat, M, N, 0, 0, cost1);
    int result2 = countPaths(mat, M, N, 0, 0, cost2);
    int result3 = countPaths(mat, M, N, 0, 0, cost3);

    std::cout << "Cost: " << cost1 << std::endl;
    std::cout << "Paths: " << result1 << std::endl;

    std::cout << "Cost: " << cost2 << std::endl;
    std::cout << "Paths: " << result2 << std::endl;

    std::cout << "Cost: " << cost3 << std::endl;
    std::cout << "Paths: " << result3 << std::endl;

    return 0;
}
```

Output	
Analysis	(Lara) This code implementation utilizes a recursive strategy with optimal substructure to divide the grid traversal into smaller subproblems. It does this by counting the valid paths from the starting cell to the bottom right in an M x N matrix, where the sum of grid values is the target cost. If the index is out of bounds, it returns 0 to remove the invalid movements outside of the matrix. To decompose this process, it recursively subtracts the current cell and branches the two valid movements, which are the right and down to return the values of rightPaths + downPaths. For the time and space complexity, it has a time complexity of $O(2^{M+N})$ for its worst case when each step branches into two recursive processes, and its space complexity is O(M+N) since it is determined by the maximum depth of traversing from 0,0 to (M-1, N-1). (Mencidor)

CONCLUSION

(LARA)

In conclusion, this Hands-on Activity has introduced the concepts of algorithmic strategies by recalling the previous algorithms used in the previous activities for review, and demonstrating the processes of these algorithms to solve problems such as the problem of finding the minimum steps needed to reduce n to 1 by using recursion. The procedures apply this starting concept with two dynamic programming techniques, which are memoization and tabulation. The differences in these methods are shown in their behavior in the code, where memoization uses recursion with caching while tabulation uses loops and arrays. Both implementations achieve the same results of solving the problem shown in the procedures. The supplementary activities expand the learnings of this part by using recursion with optimal substructure, which combines the simplicity of recursion in traversals with the optimal substructures to optimize its process in traversing parts of a matrix.

(MENCIDOR)

This activity provided a solid introduction to key algorithmic strategies by building on earlier concepts and demonstrating how recursion solves problems like finding the minimum steps to reduce N to 1. The main procedures put these ideas into practice using two dynamic programming methods: top-down memoization and bottom-up tabulation. While memoization relies on recursive calls with a memory cache and tabulation uses simple loops with an array, both approaches deliver identical optimal results. The supplementary activities further expanded on these concepts by applying recursion alongside optimal substructure, showing how combining recursive traversals with subproblem optimization simplifies and speeds up complex matrix operations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, clarification of instructions, grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain the concepts discussed in the lectures and the activity's discussion, clarify instructions, and correct grammatical errors in the analyses.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.